

```

1  #define BLYNK_PRINT Serial
2  #define BLYNK_TEMPLATE_ID "TMPL60BuceEEw"
3  #define BLYNK_TEMPLATE_NAME "u"
4  #define BLYNK_AUTH_TOKEN "s1uiK2pa0eI4sPlEp2dEzsyY7UsaHCpl"
5  #include <ESP8266WiFi.h>
6  #include <BlynkSimpleEsp8266.h>
7  #include <NTPClient.h>
8  #include <WiFiUdp.h>
9  #include <Wire.h>
10 #include <LiquidCrystal_I2C.h>
11 char ssid[] = "theecharatpondz_2.4G";
12 char pass[] = "pangpondsprite";
13 int Relay1 = D5;
14 int Relay2 = D6;
15 int Relay3 = D7;
16 int Relay4 = D8;
17 int button = D3;
18 bool relayState1 = false;
19 bool relayState2 = false;
20 bool relayState3 = false;
21 bool relayState4 = false;
22 bool lastButtonState = HIGH;
23 int soilSensorPin = A0;
24 int soilMoistureValue = 0;
25 int soilThreshold = 500;
26 bool soilControlEnable = false;
27 #define DEBOUNCE_DELAY 50
28 int64_t lastDebounceTime = 0;
29 BlynkTimer timer;
30 WiFiUDP ntpUDP;
31 NTPClient timeClient(ntpUDP, "pool.ntp.org", 7 * 3600, 60000);
32 // LCD2004 I2C
33 LiquidCrystal_I2C lcd(0x27, 20, 4);
34 // เวลาเปิด/ปิด รีเลย์
35 int r1_open_h, r1_open_m, r1_close_h, r1_close_m;
36 int r2_open_h, r2_open_m, r2_close_h, r2_close_m;
37 int r3_open_h, r3_open_m, r3_close_h, r3_close_m;
38 int r4_open_h, r4_open_m, r4_close_h, r4_close_m;
39 bool timerEnable1 = false;
40 bool timerEnable2 = false;
41 bool timerEnable3 = false;
42 bool timerEnable4 = false;
43 void setup() {
44     Serial.begin(115200);
45     pinMode(Relay1, OUTPUT);
46     pinMode(Relay2, OUTPUT);
47     pinMode(Relay3, OUTPUT);
48     pinMode(Relay4, OUTPUT);
49     pinMode(button, INPUT_PULLUP);
50     pinMode(soilSensorPin, INPUT);
51     digitalWrite(Relay1, HIGH);
52     digitalWrite(Relay2, HIGH);
53     digitalWrite(Relay3, HIGH);
54     digitalWrite(Relay4, HIGH);
55     Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
56     timeClient.begin();
57     // LCD Init
58     lcd.init();
59     lcd.backlight();
60     lcd.clear();

```

```

61     lcd.setCursor(0, 0);
62     lcd.print("Initializing...");
63     timer.setInterval(5000L, checkTimeTask);
64     timer.setInterval(5000L, checkSoilMoisture);
65     timer.setInterval(2000L, updateLCD);
66 }
67 void loop() {
68     Blynk.run();
69     timer.run();
70     timeClient.update();
71     checkButtonToggle();
72 }
73 void checkButtonToggle() {
74     bool reading = digitalRead(button);
75     int64_t now = millis();
76     if (reading != lastButtonState) {
77         lastDebounceTime = now;
78     }
79     if ((now - lastDebounceTime) > DEBOUNCE_DELAY) {
80         static bool stableState = LOW;
81         if (reading != stableState) {
82             stableState = reading;
83             if (stableState == HIGH) {
84                 relayState1 = !relayState1;
85                 digitalWrite(Relay1, relayState1 ? LOW : HIGH);
86                 Blynk.virtualWrite(V0, relayState1);
87                 Serial.printf("Relay1 (Button) %s\n", relayState1 ? "ON" : "OFF");
88             }
89         }
90     }
91     lastButtonState = reading;
92 }
93 // ปุ่มควบคุม Relay
94 BLYNK_WRITE(V0) { relayState1 = param.asInt(); digitalWrite(Relay1, relayState1 ? LOW : HIGH); }
95 BLYNK_WRITE(V1) { relayState2 = param.asInt(); digitalWrite(Relay2, relayState2 ? LOW : HIGH); }
96 BLYNK_WRITE(V2) { relayState3 = param.asInt(); digitalWrite(Relay3, relayState3 ? LOW : HIGH); }
97 BLYNK_WRITE(V3) { relayState4 = param.asInt(); digitalWrite(Relay4, relayState4 ? LOW : HIGH); }
98 BLYNK_WRITE(V6) { TimeInputParam t(param); r1_open_h = t.getStartHour(); r1_open_m = t.getStartMinute(); }
99 BLYNK_WRITE(V7) { TimeInputParam t(param); r1_close_h = t.getStartHour(); r1_close_m = t.getStartMinute(); }
100 BLYNK_WRITE(V8) { TimeInputParam t(param); r2_open_h = t.getStartHour(); r2_open_m = t.getStartMinute(); }
101 BLYNK_WRITE(V9) { TimeInputParam t(param); r2_close_h = t.getStartHour(); r2_close_m = t.getStartMinute(); }
102 BLYNK_WRITE(V10) { TimeInputParam t(param); r3_open_h = t.getStartHour(); r3_open_m = t.getStartMinute(); }
103 BLYNK_WRITE(V11) { TimeInputParam t(param); r3_close_h = t.getStartHour(); r3_close_m = t.getStartMinute(); }
104 BLYNK_WRITE(V12) { TimeInputParam t(param); r4_open_h = t.getStartHour(); r4_open_m = t.getStartMinute(); }
105 BLYNK_WRITE(V13) { TimeInputParam t(param); r4_close_h = t.getStartHour(); r4_close_m = t.getStartMinute(); }
106 BLYNK_WRITE(V14) { timerEnable1 = param.asInt(); }
107 BLYNK_WRITE(V15) { timerEnable2 = param.asInt(); }
108 BLYNK_WRITE(V16) { timerEnable3 = param.asInt(); }
109 BLYNK_WRITE(V17) { timerEnable4 = param.asInt(); }
110 BLYNK_WRITE(V4) { soilThreshold = param.asInt(); }
111 BLYNK_WRITE(V18) { soilControlEnable = param.asInt(); }
112 void checkTimeTask() {
113     timeClient.update();
114     if (!timeClient.isTimeSet()) {
115         Serial.println("Waiting for NTP time sync...");
116         return;
117     }
118     printCurrentTime();
119     checkRelayTime(Relay1, r1_open_h, r1_open_m, r1_close_h, r1_close_m, relayState1, V0, "Relay1", timerEnable1);
120     checkRelayTime(Relay2, r2_open_h, r2_open_m, r2_close_h, r2_close_m, relayState2, V1, "Relay2", timerEnable2);
121     checkRelayTime(Relay3, r3_open_h, r3_open_m, r3_close_h, r3_close_m, relayState3, V2, "Relay3", timerEnable3);
122     checkRelayTime(Relay4, r4_open_h, r4_open_m, r4_close_h, r4_close_m, relayState4, V3, "Relay4", timerEnable4);
123 }
124 void checkRelayTime(int pin, int open_h, int open_m, int close_h, int close_m, bool &state, int vpin, const char* label, bool enabled) {
125     if (!enabled || !timeClient.isTimeSet()) return;
126     int nowSec = timeClient.getHours() * 3600 + timeClient.getMinutes() * 60 + timeClient.getSeconds();
127     int startSec = open_h * 3600 + open_m * 60;
128     int stopSec = close_h * 3600 + close_m * 60;
129     if (nowSec >= startSec && nowSec <= stopSec) {
130         if (!state) {
131             state = true;

```

```

132     digitalWrite(pin, LOW);
133     Blynk.virtualWrite(vpin, 1);
134     Serial.printf("%s (Time) ON\n", label);
135 }
136 } else {
137     if (state) {
138         state = false;
139         digitalWrite(pin, HIGH);
140         Blynk.virtualWrite(vpin, 0);
141         Serial.printf("%s (Time) OFF\n", label);

```

```

142     }
143 }
144 }
145 void printCurrentTime() {
146     Serial.printf("Current Time (NTP): %02d:%02d:%02d\n", timeClient.getHours(), timeClient.getMinutes(), timeClient.getSeconds());
147 }
148 void checkSoilMoisture() {
149     soilMoistureValue = analogRead(soilSensorPin);
150     int moisturePercent = map(soilMoistureValue, 1023, 300, 0, 100);
151     moisturePercent = constrain(moisturePercent, 0, 100);
152     Serial.print("Soil moisture (%): ");
153     Serial.println(moisturePercent);
154     Serial.print("Soil moisture : ");
155     Serial.println(soilMoistureValue);
156     if (soilControlEnable) {
157         if (moisturePercent < soilThreshold && !relayState1) {
158             relayState1 = true;
159             digitalWrite(Relay1, LOW);
160             Blynk.virtualWrite(V0, 1);
161             Serial.println("Relay1 (Soil) ON");
162         } else if (moisturePercent >= soilThreshold && relayState1) {
163             relayState1 = false;
164             digitalWrite(Relay1, HIGH);
165             Blynk.virtualWrite(V0, 0);
166             Serial.println("Relay1 (Soil) OFF");
167         }
168     }
169 }
170 BLYNK_WRITE(V19) {
171     int btnState = param.asInt();
172     if (btnState == 1) {
173         soilMoistureValue = analogRead(soilSensorPin);
174         int moisturePercent = map(soilMoistureValue, 1023, 300, 0, 100);
175         Blynk.virtualWrite(V5, moisturePercent);
176         Serial.printf("ส่งค่า Soil ไป V5: %d\n", moisturePercent);
177         Serial.printf("ส่งค่า Soil ไป V5: %d\n", moisturePercent);
178     }
179 }
180 void updateLCD() {
181     lcd.clear();
182     lcd.setCursor(0, 0);
183     lcd.printf("Time: %02d:%02d:%02d", timeClient.getHours(), timeClient.getMinutes(), timeClient.getSeconds());
184     int moisturePercent = map(soilMoistureValue, 1023, 300, 0, 100);
185     moisturePercent = constrain(moisturePercent, 0, 100);
186     lcd.setCursor(0, 1);
187     lcd.printf("Soil: %3d %%", moisturePercent);
188     lcd.setCursor(0, 2);
189     lcd.printf("R1:%s R2:%s", relayState1 ? "ON " : "OFF", relayState2 ? "ON " : "OFF");
190     lcd.setCursor(0, 3);
191     lcd.printf("R3:%s R4:%s", relayState3 ? "ON " : "OFF", relayState4 ? "ON " : "OFF");

```